

Ejercicios

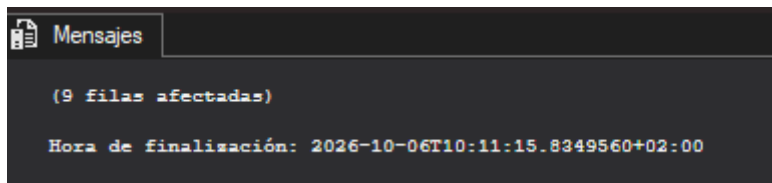
Ejercicio1

consulta

```
DROP TABLE IF EXISTS dbo.artistas_copia;
```

```
SELECT *  
INTO dbo.artistas_copia  
FROM dbo.artistas;
```

Salida



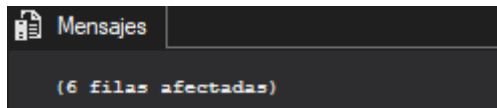
Ejercicio 2

Consulta

```
DROP TABLE IF EXISTS dbo.usuarios_latam;

SELECT usuario_id, nombre_usuario, pais, plan_suscripcion
INTO dbo.usuarios_latam
FROM dbo.usuarios
WHERE pais IN ('MX', 'AR', 'CO', 'PR');
```

Resultado



Consulta ordenada del resultado

```
SELECT TOP (1000) [usuario_id]
, [nombre_usuario]
, [pais]
, [plan_suscripcion]
FROM [SonoraDB].[dbo].[usuarios_latam]
ORDER BY pais, nombre_usuario
```

Resultado

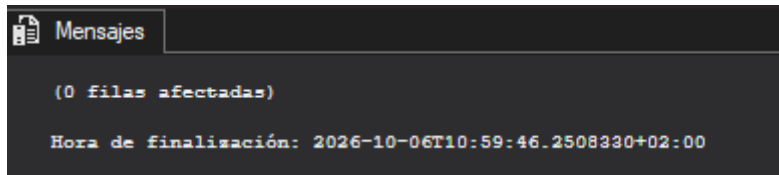
	usuario_id	nombre_usuario	pais	plan_suscripcion
1	4	sofi_trap	AR	Familiar
2	8	vale_indie	AR	Premium
3	5	dani_dj	CO	Free
4	3	juanpi	MX	Premium
5	7	nachox	MX	Free
6	10	kiara_perreo	PR	Premium

Ejercicio 3

Consulta para crear la tabla

```
DROP TABLE IF EXISTS dbo.stg_usuarios_2309
select *
into dbo.stg_usuarios_2309
from dbo.usuarios
where 1=0;
```

salida



comprobación que no hay filas

```
SELECT *
FROM dbo.stg_usuarios_2309;
```

Salida

	usuario_id	nombre_usuario	email	pais	plan_suscripcion	fecha_alta
--	------------	----------------	-------	------	------------------	------------

Comprobación de la estructura

```
SELECT *
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'stg_usuarios_2309'
AND TABLE_SCHEMA = 'dbo';
```

Resultado

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE
1	SonoraDB	dbo	stg_usuarios_2309	usuario_id	1	NULL	NO
2	SonoraDB	dbo	stg_usuarios_2309	nombre_usuario	2	NULL	NO
3	SonoraDB	dbo	stg_usuarios_2309	email	3	NULL	NO
4	SonoraDB	dbo	stg_usuarios_2309	pais	4	NULL	NO
5	SonoraDB	dbo	stg_usuarios_2309	plan_suscripcion	5	NULL	NO
6	SonoraDB	dbo	stg_usuarios_2309	fecha_alta	6	NULL	NO

Se podría insertar dos veces una fila con usuario_id=1 si se podría ya que en una cta no se compian las claves primarias ni las restricciones que tuviera la tabla de

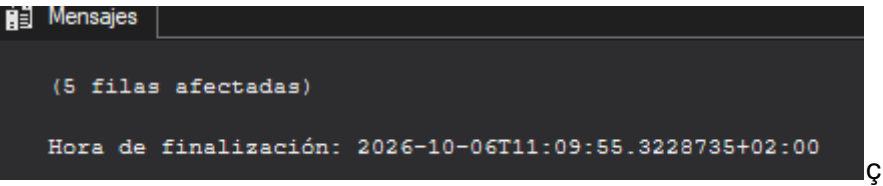
origen y una fila con email en null la respuesta a esta pregunta es que depende ya que todo depende de si la tabla de origen permitía nulos o no ya que eso si se hereda

Ejercicio 4

Creación de la tabla

```
DROP TABLE IF EXISTS dbo.anuncios_emitidos
select reproduccion_id, usuario_id , fecha_hora , dispositivo
into dbo.anuncios_emitidos
from dbo.reproducciones
where tipo_contenido = N'Anuncio';
```

resultado



Muestra ordenada

```
select *
from dbo.anuncios_emitidos
order by reproduccion_id
```

salida

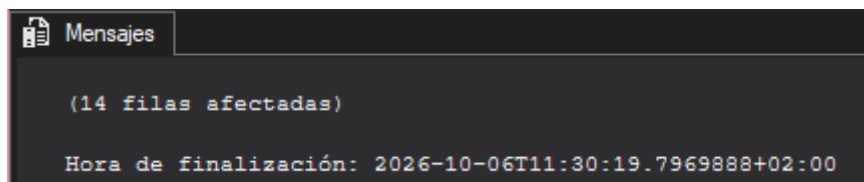
	reproduccion_id	usuario_id	fecha_hora	dispositivo
1	5	2	2026-09-14 10:06:00	Web
2	12	5	2026-09-15 21:00:00	Móvil
3	19	7	2026-09-16 16:24:00	Móvil
4	32	2	2026-09-20 18:00:00	Web
5	37	5	2026-09-21 22:00:00	Móvil

Ejercicio 5

Consulta corregida

```
SELECT
    u.pais AS pais_usuario,
    a.pais AS pais_artista,
    COUNT(*) AS total_reproducciones
INTO dbo.cruce_paises
FROM dbo.reproducciones AS r
JOIN dbo.usuarios AS u ON u.usuario_id = r.usuario_id
JOIN dbo.canciones AS c ON c.cancion_id = r.cancion_id
JOIN dbo.artistas AS a ON a.artista_id = c.artista_id
WHERE r.tipo_contenido = N'Canción'
    AND r.segundos_escuchados >= 30
GROUP BY u.pais, a.pais;
```

Resultado



Los problemas presentes son que las tablas no tenían alias por lo que había un problema de duplicados y el segundo problema presente es que las columnas calculadas en una cta exigen tener un alias obligatorio

Ejercicio 6

Creación de la tabla

```
drop table if exists dbo.canciones_ficha

select c.cancion_id, c.titulo,
cast(c.duracion_seg / 60.0 as decimal(4,1))as duracion_min,
cast( case when c.duracion_seg >= 300 then 'Larga'
          when c.duracion_seg between 180 and 299 then 'Media'
          else 'Corta'
        END AS varchar(10)) AS categoria

into dbo.canciones_ficha
from dbo.canciones as c
```

salida

```
(15 filas afectadas)

Hora de finalización: 2026-10-06T12:31:30.4625107+02:00
```

Comprobación de los tipos creados

```
SELECT *
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'canciones_ficha'
AND TABLE_SCHEMA = 'dbo';
```

resultado

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LENGTH	CHARACTER_OCTET_LENGTH	NUMERIC_PRECISION
1	SonoraDB	dbo	canciones_ficha	cancion_id	1	NULL	NO	int	NULL	NULL	10
2	SonoraDB	dbo	canciones_ficha	titulo	2	NULL	NO	nvarchar	150	300	NULL
3	SonoraDB	dbo	canciones_ficha	duracion_min	3	NULL	YES	decimal	NULL	NULL	4
4	SonoraDB	dbo	canciones_ficha	categoria	4	NULL	YES	varchar	10	10	NULL

Cuenta de las canciones de cada categoría

```
select categoria, count(categoria) as numerocanciones
from dbo.canciones_ficha
group by categoria
order by categoria asc
```

resultado

Resultados		Mensajes
	categoria	numerocanciones
1	Corta	4
2	Larga	4
3	Media	7

Ejercicio 7

Creación de la tabla

```
DROP TABLE IF EXISTS dbo.seguimiento_lanzamientos;

SELECT  cancion_id,
        titulo,
        fecha_lanzamiento,
        CAST(N'Sin revisar' AS nvarchar(30)) AS estado,
        CAST(NULL AS nvarchar(100)) AS revisado_por,
        CAST(NULL AS date) AS fecha_revision
INTO dbo.seguimiento_lanzamientos
FROM dbo.canciones
WHERE fecha_lanzamiento >= '2026-08-01';
GO
```

(3 filas afectadas)

Hora de finalización: 2026-10-06T12:38:10.8999605+02:00

Primera revisión

```
UPDATE dbo.seguimiento_lanzamientos
SET estado = N'Revisado: apta para portada',
    revisado_por = N'Carmen Lozano',
    fecha_revision = '2026-09-23'
WHERE titulo = N'Galaxia Dembow';
GO
```

(1 fila afectada)

Hora de finalización: 2026-10-06T12:38:45.7149949+02:00

Tabla ordenada por fecha de lanzamiento

```
SELECT  cancion_id,
        titulo,
        fecha_lanzamiento,
        estado,
        revisado_por,
        fecha_revision
FROM dbo.seguimiento_lanzamientos
ORDER BY fecha_lanzamiento ASC;
```

Resultado

	cancion_id	titulo	fecha_lanzamiento	estado	revisado_por	fecha_revision
1	108	Humo Violeta	2026-08-22	Sin revisar	NULL	NULL
2	113	Galaxia Dembow	2026-09-04	Revisado: apta para portada	Carmen Lozano	2026-09-23
3	114	Sin Cobertura	2026-09-18	Sin revisar	NULL	NULL

Ejercicio 8

Consulta

```
DROP TABLE IF EXISTS dbo.catalogo_completo;

SELECT c.cancion_id,
       c.titulo,
       a.nombre AS artista,
       a.pais AS pais_artista,
       g.nombre AS genero,
       gp.nombre AS genero_padre
INTO dbo.catalogo_completo
FROM dbo.canciones AS c
JOIN dbo.artistas AS a ON a.artista_id = c.artista_id
JOIN dbo.generos AS g ON g.genero_id = c.genero_id
JOIN dbo.generos AS gp ON gp.genero_id = g.genero_padre_id;
GO
```

Resultado

```
(15 filas afectadas)
stg_reproducciones
Hora de finalización: 2026-10-06T13:11:20.8392940+02:00
```

Contado de las canciones por subgénero

```
SELECT genero_padre,
       COUNT(*) AS canciones
FROM dbo.catalogo_completo
GROUP BY genero_padre
ORDER BY canciones DESC, genero_padre ASC;+
```

Salida

	genero_padre	canciones
1	Urbano	6
2	Electrónica	3
3	Rock	3
4	Música	2
5	Reguetón	1

Ejercicio 9

Creación de la tabla

```
DROP TABLE IF EXISTS dbo.actividad_usuarios;

SELECT
    usu.usuario_id,
    usu.nombre_usuario,
    usu.plan_suscripcion,
    COUNT(CASE WHEN rep.tipo_contenido = 'Canción' AND rep.segundos_escuchados
    >= 30 THEN 1 END) AS validas,
    COUNT(CASE WHEN rep.tipo_contenido = 'Canción' AND rep.segundos_escuchados <
    30 THEN 1 END) AS saltos,
    COUNT(CASE WHEN rep.tipo_contenido = 'Anuncio' THEN 1 END ) AS Anuncios
INTO dbo.actividad_usuarios
FROM dbo.usuarios AS usu
LEFT JOIN dbo.reproducciones AS rep ON rep.usuario_id = usu.usuario_id
GROUP BY
    usu.usuario_id,
    usu.nombre_usuario,
    usu.plan_suscripcion;
```

resultado

```
Advertencia: valor NULL eliminado por el agregado u otra operación SET.

(10 filas afectadas)
```

Mostrado

```
SELECT [usuario_id]
      ,[nombre_usuario]
      ,[plan_suscripcion]
      ,[validas]
      ,[saltos]
      ,[Anuncios]
FROM [SonoraDB].[dbo].[actividad_usuarios]
order by usuario_id
```

resultado

	usuario_id	nombre_usuario	plan_suscripcion	validas	saltos	Anuncios
1	1	alexbeats	Premium	6	0	0
2	2	marta_rock	Free	4	0	2
3	3	juanpi	Premium	6	0	0
4	4	sofi.trap	Familiar	4	2	0
5	5	dani_dj	Free	3	0	2
6	6	lucia.m	Premium	4	1	0
7	7	nachox	Free	3	0	1
8	8	vale_indie	Premium	0	0	0
9	9	pau.synth	Free	0	0	0
10	10	kiara_perreo	Premium	0	0	0

Ejercicio 10

Cte recursiva

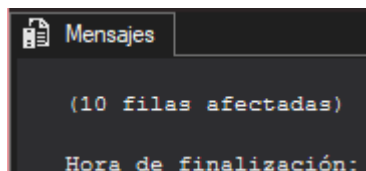
```
DROP TABLE IF EXISTS dbo.organigrama_plano;

WITH arbol AS (
    -- Caso base el CEO quien no tiene jefe
    SELECT empleado_id, nombre, puesto, jefe_id, 0 AS nivel,
           CAST(nombre AS nvarchar(500)) AS ruta
    FROM dbo.empleados
    WHERE jefe_id IS NULL

    UNION ALL

    -- Caso recursivo
    SELECT e.empleado_id, e.nombre, e.puesto, e.jefe_id, a.nivel + 1,
           CAST(a.ruta + N' > ' + e.nombre AS nvarchar(500))
    FROM dbo.empleados AS e
    JOIN arbol AS a ON e.jefe_id = a.empleado_id
)
SELECT empleado_id, nombre, puesto, nivel, ruta
INTO dbo.organigrama_plano
FROM arbol;
```

Resultado



Cuenta los empleados

```
SELECT nivel,
       COUNT(*) AS empleados
FROM dbo.organigrama_plano
```

```
GROUP BY nivel
ORDER BY nivel ASC;
```

Resultado

	nivel	empleados
1	0	1
2	1	2
3	2	3
4	3	4

Lista con like

```
SELECT nombre, puesto, nivel
FROM dbo.organigrama_plano
WHERE ruta LIKE N'%Raúl Benet%'
ORDER BY nivel ASC, nombre ASC;
```

Resultado

	nombre	puesto	nivel
1	Raúl Benet	Head of Data	2
2	Carmen Lozano	Data Analyst	3
3	Elena Ruiz	Data Engineer	3
4	Pablo Ortega	Data Engineer	3

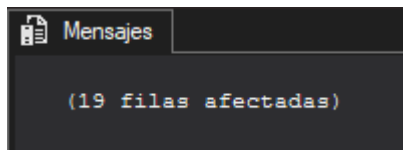
Ejercicio 11

Creación de la tabla

```
DROP TABLE IF EXISTS dbo.personas_por_pais;

SELECT pais,
       CAST(N'Usuario' AS nvarchar(10)) AS tipo,
       nombre_usuario AS nombre
INTO dbo.personas_por_pais
FROM dbo.usuarios
UNION ALL
SELECT pais,
       CAST(N'Artista' AS nvarchar(10)) AS tipo,
       nombre
FROM dbo.artistas;
```

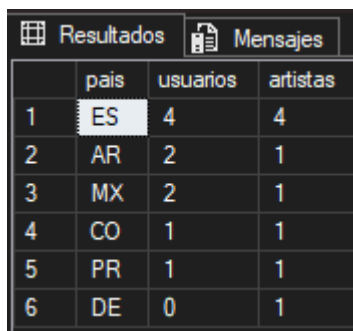
Resultado



Mostar usuarios y clientes por pais

```
SELECT pais,  
       SUM(CASE WHEN tipo = N'Usuario' THEN 1 ELSE 0 END) AS usuarios,  
       SUM(CASE WHEN tipo = N'Artista' THEN 1 ELSE 0 END) AS artistas  
FROM dbo.personas_por_pais  
GROUP BY pais  
ORDER BY usuarios DESC, pais ASC;
```

Resultado



A screenshot of the SQL Server Results window. The title bar shows a grid icon and the word 'Resultados'. The main area displays a table with 6 rows and 4 columns: 'pais', 'usuarios', and 'artistas'. The first row is highlighted.

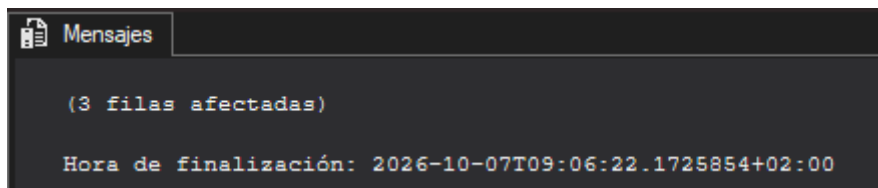
	pais	usuarios	artistas
1	ES	4	4
2	AR	2	1
3	MX	2	1
4	CO	1	1
5	PR	1	1
6	DE	0	1

Ejercicio 12

Creación de la foto

```
DROP TABLE IF EXISTS dbo.foto_usuarios_plan;
```

```
SELECT plan_suscripcion,  
       COUNT(*) AS usuarios  
INTO dbo.foto_usuarios_plan  
FROM dbo.usuarios  
GROUP BY plan_suscripcion;
```



Mostar planes

```
BEGIN TRAN;

UPDATE dbo.usuarios
SET plan_suscripcion = N'Premium'
WHERE nombre_usuario = N'nachox';

SELECT COALESCE(f.plan_suscripcion, a.plan_suscripcion) AS plan_suscripcion,
       f.usuarios AS usuarios_foto,
       a.usuarios AS usuarios_ahora
FROM dbo.foto_usuarios_plan AS f
FULL JOIN (
    SELECT plan_suscripcion, COUNT(*) AS usuarios
    FROM dbo.usuarios
    GROUP BY plan_suscripcion
) AS a ON a.plan_suscripcion = f.plan_suscripcion
WHERE COALESCE(f.usuarios, 0) <> COALESCE(a.usuarios, 0);

ROLLBACK;
```

Resultado

	plan_suscripcion	usuarios_foto	usuarios_ahora
1	Free	4	3
2	Premium	5	6

Ejercicio 13

Creación tabla

```
DROP TABLE IF EXISTS dbo.ranking_artistas_nueva;

SELECT a.artista_id,
       a.nombre AS artista,
       COUNT(r.reproduccion_id) AS reproducciones_validas,
       COUNT(DISTINCT r.usuario_id) AS oyentes
INTO dbo.ranking_artistas_nueva
FROM dbo.artistas AS a
JOIN dbo.canciones AS c ON c.artista_id = a.artista_id
JOIN dbo.reproducciones AS r ON r.cancion_id = c.cancion_id
WHERE r.tipo_contenido = N'CanCIÓN'
      AND r.segundos_escuchados >= 30
GROUP BY a.artista_id, a.nombre;

ALTER TABLE dbo.ranking_artistas_nueva
ALTER COLUMN artista_id int NOT NULL;

SELECT (SELECT COUNT(*) FROM dbo.ranking_artistas_nueva) AS filas_nueva,
       (SELECT COUNT(*) FROM (
           SELECT artista_id
           FROM dbo.ranking_artistas_nueva
```

```

        GROUP BY artista_id
        HAVING COUNT(*) > 1
    ) AS repetidos) AS ids_repetidos;

BEGIN TRAN;

DROP TABLE IF EXISTS dbo.ranking_artistas;

EXEC sp_rename N'dbo.ranking_artistas_nueva', N'ranking_artistas';

ALTER TABLE dbo.ranking_artistas
ADD CONSTRAINT PK_ranking_artistas PRIMARY KEY (artista_id);

COMMIT;

SELECT artista,
       reproducciones_validas,
       oyentes
FROM dbo.ranking_artistas
ORDER BY reproducciones_validas DESC, artista ASC;

```

	filas_nueva	ids_repetidos
1	8	0

	artista	reproducciones_validas	oyentes
1	MC Brisa	11	5
2	Los Voltios	5	2
3	Luna Roja	3	2
4	Nébula	3	3
5	DJ Coral	2	2
6	Hierro Fundido	2	2
7	Kaiser Beats	2	2
8	Verso Libre	2	2